IT403-P | GROUP PRE-EXAMINATION ACTIVITY | 30 POINTS

Security Feature Finalization & Change Review

CAPSTONE 36 OF 38 — LABSYNC

Capstone information	Group information
Capstone title: LabSync: An IoT-Based IT Laboratory Availability and Equipment Monitoring System Using QR Codes at Bulacan State University – Sarmiento Campus **Date:** 10/18/2026	**Group / Section:** GROUP 3 – BSIT 4E2 **Group leader:** Gabito, Andrei A.

Activity purpose and instructions

Review the original security-feature proposal and finalize the security controls to be examined. This activity is graded separately (30 points); the final examination remains 100 points.

1. **List and classify.** Include every originally proposed security feature and any new requested feature. Mark each as RETAIN, REMOVE, or ADD.
2. **Justify and verify.** Give a brief reason for every decision, particularly additions and removals. Describe a practical verification test or evidence reference.
3. **Obtain approval.** The instructor marks INCLUDE or EXCLUDE for each feature. An unapproved removal remains required; an unapproved addition is not assessed.

Consolidated security feature inventory and revision requests

Prelisted entries summarize previously proposed controls for review; they are NOT confirmed implemented or approved. Mark each RETAIN or REMOVE and justify the decision. Use the ADD rows for new features. Any entry marked [CONFIRM SCOPE] needs clarification before instructor approval.

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
LOGIN AND ACCOUNT SECURITY						
1	**Account provisioning and status checks:** Server authentication through express-session/requireAuth.	**RETAIN**	**We retain this feature.** In our system, `express-session` verifies every staff route on the server side. During login, our `authService.js` checks if the user account is active (`Is_Active = 1`) and verifies whether an OJT student's internship has concluded. In addition, our `requireRole` middleware checks the database on every request so deactivated users are immediately cut off. We also clarify that public student PC fault reporting is intentionally exempt from login sessions so students can easily report broken PCs at lab kiosks without needing an account.	**Planned Live Test:** 1. Open a new Incognito window and try navigating directly to any protected staff page (e.g. `/mis-keys.html` or `/faculty-management.html`); observe that the server immediately blocks access and redirects back to the login screen with an unauthorized warning. 2. Log in as an active staff user, then have an administrator deactivate that account in Faculty Management; clicking any dashboard button immediately ends the session and displays an "Account Deactivated" security alert.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
2	**Credential recovery and updates:** cryptographically random 32-byte reset tokens with expiry.	**RETAIN**	**We retain this feature.** For self-service password recovery, our backend generates a cryptographically secure 32-byte random token using `crypto.randomBytes(32)`. To protect our database, we only save the SHA-256 hash of the token in the `users` table with a strict 1-hour expiration. Once used, the token is reset to `NULL` so it cannot be reused, and our API returns a generic success message so an attacker cannot enumerate valid school emails.	**Planned Live Test:** 1. On the login page, click "Forgot Password?", enter a registered BulSU faculty email, and click Send. Open the email to confirm receipt of a secure recovery link containing a 64-character hexadecimal token. 2. Click the link and enter a new password to successfully reset your account. Then, immediately click the exact same recovery link again; observe that the system rejects the request as expired or already consumed (single-use token guarantee).		
3	**Secure password handling:** password hashes identified as assets, with no algorithm specified.	**RETAIN**	**We retain this feature and specify our exact algorithm as bcrypt with 12 salt rounds.** This resolves our instructor's review flag regarding the unspecified algorithm. In our `authService.js`, passwords are never stored in plaintext; we hash them using the `bcrypt` library (^6.0.0) with salt cost factor 12 (`BCRYPT_SALT_ROUNDS = 12`). Password hashes are strictly excluded from all API responses and audit logs across all user roles.	**Planned Live Test:** 1. Open phpMyAdmin or run a query on the MySQL `users` table to visually inspect password records; confirm they are stored as one-way hashed strings starting with `$2b$12$` (identifying bcrypt with 12 salt rounds) rather than plain text. 2. On the login screen, enter the correct password to demonstrate successful sign-in; then change a single letter in the password and re-submit to demonstrate that bcrypt verification safely rejects the attempt with "Invalid credentials".		
SESSION SECURITY						
4	**Session identifiers and renewal:** Server session checks.	**RETAIN**	**We retain this feature to prevent session fixation and hijacking.** Whenever a user logs in successfully, our `auth.controller.js` runs `req.session.regenerate()` to issue a brand new session ID. When logging out via `POST /api/logout`, our backend explicitly runs `req.session.destroy()` and clears the `connect.sid` cookie, ensuring old or stolen session IDs cannot be used to access protected data.	**Planned Live Test:** 1. In Google Chrome or Edge, press F12, open the Application tab, select Cookies, and check the current `connect.sid` cookie value before logging in. 2. Enter credentials and click Sign In; observe in DevTools that `connect.sid` instantly regenerates to a brand-new alphanumeric ID (stopping session fixation attacks). 3. Click "Log Out" in the top bar; observe that `connect.sid` is completely deleted from the browser, and clicking the browser Back button will not reopen the dashboard.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
5	**Session and cookie protection:** protected Secure/HttpOnly cookies; client auth guards as supplemental interface checks.	**RETAIN**	**We retain this feature.** In our `server.js`, session cookies are configured with `httpOnly: true` (so client scripts cannot steal them via XSS) and `sameSite: 'lax'`. On our live Railway deployment, we enable `secure: true` with reverse proxy support. We clarify that our client-side `auth-check.js` is only an anti-flash UI guard to prevent unauthorized page rendering, while our Express backend independently verifies every API call.	**Planned Live Test:** 1. While signed in, open the browser DevTools Console (F12) and type `document.cookie` then press Enter. Notice that `connect.sid` is completely blank/hidden from JavaScript, proving malicious XSS scripts cannot steal the session cookie. 2. In the DevTools Application > Cookies tab, verify that the `HttpOnly`, `Secure` (HTTPS), and `SameSite=Lax` columns all show checkmarks.		
6	**Session expiry and revocation:** HTTPS/TLS for sensitive traffic.	**RETAIN**	**We retain this feature.** We set a 10-minute inactivity timeout in our `auth.js` middleware so that if a user leaves the computer unattended, the session is destroyed (background polling with `X-Background-Poll: true` is ignored so it does not falsely keep the session alive). We also enforce a 24-hour max cookie lifetime, and our production Railway deployment enforces HTTPS/TLS with 180-day HSTS security headers.	**Planned Live Test:** 1. Log in to the MIS dashboard and leave the computer untouched for 10 minutes (simulating leaving a lab PC unattended). Click any navigation link or table action; the system automatically terminates the session and returns you to the login screen with a "Session Expired" notification. 2. Click the padlock icon in the browser address bar to verify the active SSL/TLS certificate and HSTS encryption enforcing secure HTTPS connections.		
ACCESS CONTROL						
7	**Role-based authorization:** Faculty, IT Head and MIS Staff RBAC; protected API checks.	**RETAIN**	**We retain this feature.** Our backend enforces role-based access control across our 4 system roles: IT Head / Program Coordinator, MIS Staff, Faculty, and OJT. Our `requireRole` middleware checks the database on every request to confirm the user's live role. Key reservations for a 2nd room can only be approved by the IT Head. OJT accounts are blocked from managing keys or withdrawing physical keys. If someone bypasses the UI and calls an API directly, our server returns HTTP 403.	**Planned Live Test:** 1. Log in with a Faculty account: observe that administrative menus (such as MIS Staff and Master Schedule Locks) are completely hidden from the sidebar. 2. Type the URL `mis-staff.html` directly in the address bar; the server immediately blocks the page and displays "Access Forbidden (HTTP 403)". 3. Sign in as an OJT account to confirm that physical laboratory key dispatch controls and sensitive administrative actions are disabled.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
8	**Privileged actions and account governance:** public student fault-reporting flow; restricted account and schedule management.	**RETAIN**	**We retain this feature.** We separate public student fault reporting from privileged staff functions. Students can report broken PCs on public endpoints protected by rate limiting and a 30-minute student cooldown. All administrative features (managing faculty, creating accounts, locking schedules) strictly require IT Head or MIS Staff roles. We also disable hard deletion of staff accounts so our audit history is never broken.	**Planned Live Test:** 1. On a public lab kiosk or mobile phone, open the student fault reporting form without logging in; verify that scanning a PC QR code and submitting a hardware issue succeeds without prompting for a staff login. 2. Open an Incognito window and attempt to access `faculty-management.html` or delete a maintenance report; observe that the server blocks the action with HTTP 401 Unauthorized, proving public and administrative privileges are strictly isolated.		
INPUT AND QUERY SECURITY						
9	**Input validation and safe transaction processing:** Server validation of public-form data; frontend validation and clear form labels.	**RETAIN**	**We retain this feature.** In our backend services, we validate all incoming form data, including laboratory room numbers, PC numbers, allowed issue categories, and text lengths. For multi-step actions like submitting a maintenance report or transferring key custody, we wrap them in database transactions (`withTransaction`) so that if any single step fails, the entire operation safely rolls back.	**Planned Live Test:** 1. Open the PC fault report form and attempt to submit with required fields left blank or an impossible PC number (e.g., PC-999); observe immediate red client-side validation banners preventing submission. 2. Inspect the network response to confirm that our server also validates input independently, returning HTTP 400 Bad Request if invalid payload data is sent, preventing any corrupted records in MySQL.		
10	**Database and injection protections:** parameterized queries and least-privilege database access.	**RETAIN**	**We retain this feature.** All SQL queries in our repositories use parameterized queries (`?` placeholders) through `mysql2`, completely preventing SQL injection by treating all inputs strictly as data values. We keep our database credentials secure in our private `.env` file, and table structures are managed cleanly during startup through `migrate.js`.	**Planned Live Test:** On the login form or dashboard search input, type standard SQL injection strings such as `' OR '1'='1` or `admin'--` and submit. Observe that the database treats the input strictly as literal string text, returning "Invalid email or password" or "No records found" with zero SQL syntax errors and without bypassing authentication.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
DATA PROTECTION AND PRIVACY						
11	**Sensitive data encryption and key protection:** Protected .env database/email secrets excluded from version control.	**RETAIN**	**We retain this feature.** We protect all our system secrets by loading database passwords, session secrets, and IoT device tokens strictly from environment variables. We placed `.env` and `LabSync_ESP32/secrets.h` in `.gitignore` so they are never committed to GitHub. Our `validateProductionConfig()` halts server startup if required secrets are missing, and our audit logger automatically strips secret tokens.	**Planned Live Test:** 1. Open our public GitHub repository in a browser and verify that `.env` and `LabSync_ESP32/secrets.h` do not exist in the file tree, proving credentials are never committed. 2. Open the Railway live project dashboard and click on the "Variables" tab to show that database passwords and session encryption keys are securely injected into the container environment at runtime.		
12	**Personal-data minimization and lifecycle:** no sensitive details in URLs, logs or errors.	**RETAIN**	**We retain this feature.** In production, our `errorHandler.js` returns a generic `"Internal server error"` to the client, preventing stack traces or database schema leaks. For privacy compliance, our `activityRetentionService.js` runs a daily background task that automatically deletes `occupancy_log` records older than 1 calendar year using indexed SQL queries.	**Planned Live Test:** 1. In the browser, navigate to an invalid route or deliberately trigger a server error; observe that the screen displays a safe, generic error message without revealing internal source code paths, SQL queries, or server stack traces. 2. Open the Activity Log table on the MIS dashboard to show that historical log records older than 1 calendar year are automatically cleaned up in accordance with privacy compliance.		
DEVICE AND MESSAGE SECURITY						
13	**Device identity and authorized association:** Authenticated JSON payloads with device security tokens in stakeholder responsibilities.	**RETAIN**	**We retain this feature.** Our ESP32 key dock connects to our backend using an `Authorization: Bearer <token>` header. When IoT authentication is enabled, `requireIoTAuth` checks the token against our `iot_devices` table. In addition, our backend checks `authorizedRooms` to make sure the device can only report key events for its designated laboratory room (Lab 203 and 204), rejecting other rooms with HTTP 403.	**Planned Live Test:** 1. Power on the physical ESP32 key dock; observe on the OLED display and serial output that it performs an authenticated handshake with the Railway server using its pre-registered Bearer token. 2. Demonstrate server enforcement by showing that any request sent with an invalid device token or targeting an unauthorized laboratory room returns HTTP 401 Unauthorized or HTTP 403 `IOT_UNAUTHORIZED_ROOM`.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
14	**Hardware resilience and monitoring:** campus Wi-Fi; 100-ms sensor debouncing; Wi-Fi reconnection and hardware heartbeats.	**RETAIN**	**We retain this feature with our actual hardware parameters.** In our ESP32 firmware, we use a **40-millisecond debounce** (`delay(40)`) with 16-sample ADC averaging on pins D32 and D33. This filters out electrical contact noise when physical keys are pulled or inserted. The ESP32 sends a telemetry heartbeat every 15 seconds, automatically reconnects to Wi-Fi if the signal drops, and saves up to 16 pending security alerts in NVS flash memory.	**Planned Live Test:** 1. At the physical key dock, rapidly insert or pull a key; observe that the 40ms ADC debouncing smoothly registers clean state transitions without false duplicate key triggers. 2. Turn off the lab Wi-Fi hotspot; observe the ESP32 enter automatic reconnection mode while safely buffering event logs in non-volatile flash memory, then syncing them immediately when Wi-Fi reconnects.		
AUDIT LOGGING AND SAFE ERRORS						
15	**Security and activity audit trail:** occupancy_log for QR/key events; activity feed plus login, authorization, validation and administrative security events.	**RETAIN**	**We retain this feature for complete accountability.** We built a dual-table logging system: `occupancy_log` records physical key events (`Key Taken`, `Key Returned`, `UNAUTHORIZED`, `WRONG_SLOT`), while `audit_logs` records administrative events (`LOGIN`, `LOGOUT`, `PASSWORD_RESET`, `IOT_AUTH_FAILURE`, schedule changes) with user IDs, IP addresses, and timestamps. We have no delete endpoints for these tables.	**Planned Live Test:** 1. At the physical ESP32 dock, pull a laboratory key without scanning a faculty QR authorization; the dock buzzer immediately sounds an audible alarm. 2. Open the MIS Staff "Activity Logs" page on screen; observe the new entry showing `IOT_UNAUTHORIZED_KEY_REMOVAL` with the exact timestamp, room number, slot index, and flagged alert badge.		
AVAILABILITY BACKUP AND RECOVERY						
16	**Service continuity and availability:** MySQL2 connection pooling and asynchronous error handling.	**RETAIN**	**We retain this feature with clarified scope.** As our instructor noted, connection pooling and error handling manage resources and keep our application running smoothly—they do not guarantee 99.9% cloud uptime (which is managed by Railway). Our backend uses a 10-connection MySQL pool with keep-alive, centralized async error middleware to prevent server crashes, and `/api/health` and `/api/health/db` endpoints to monitor operational status.	**Planned Live Test:** Open any web browser and visit `https://labsync-bulsu-sarmiento-production.up.railway.app/api/health` and `/api/health/db`. Observe that the server immediately returns a clean JSON health badge: `{"status":"ok", "database":"connected"}`, proving operational uptime and active MySQL connection pool health.		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
17	**Backup and recovery controls:** regular backups.	**REMOVE**	**We decided to remove this from our software code requirements and reclassify it as cloud platform infrastructure.** In our codebase audit, we verified that our repository does not have automated backup scripts or cron jobs in the application code. Instead, automated database snapshots and point-in-time recovery are managed by Railway's cloud infrastructure. Removing this from the code scope keeps our capstone implementation accurate and honest.	**Planned Live Test:** Log in to the Railway cloud platform dashboard (railway.app) and open the LabSync MySQL service. Click the "Backups" tab to show the active automated volume snapshot timeline, automated daily backup schedule, and one-click database restore capability.		
18	**Secure maintenance and operational checks:** three-second dashboard polling and hardware diagnostics.	**RETAIN**	**We retain this feature with clarified wording.** The 3-second polling on our MIS keys dashboard is an interface auto-refresh so staff can monitor live key custody without refreshing the page. Our hardware diagnostics consist of passive telemetry sent by our ESP32 (slot states, free heap memory, and online status via `/api/occupancy/heartbeat`), rather than active hardware self-test commands.	**Planned Live Test:** Open `mis-keys.html` on the MIS dashboard in a browser. At the physical dock, take or return a laboratory key; without touching or refreshing the browser, watch the key status card on the screen automatically change between "Available" and "In Use" within 3 seconds, driven by live background polling.		
ADDITIONAL SECURITY FEATURES (OPTIONAL — COMPLETE ONLY IF PROPOSING ADDITIONS)						
A1	**Student ID QR Cryptographic Verification & Replay Protection:** Ephemeral HMAC SHA-256 capture tokens with single-use nonce consumption.	**ADD**	**We are adding this feature to directly solve our instructor's review flag regarding QR token expiry and replay prevention.** In `studentVerificationService.js`, when a student scans their physical BulSU ID barcode/QR, our server creates an ephemeral token signed with HMAC SHA-256 containing a 16-byte random nonce, 10-minute expiry, and room binding. Upon submitting the report, our database atomically marks the token as used (`Used_At = NOW()`) inside a transaction, preventing any replay or reuse.	**Planned Live Test:** 1. At the student kiosk, scan a valid BulSU student ID barcode to unlock the PC fault reporting form, fill in the problem details, and click Submit (report succeeds). 2. Click the browser Back button and attempt to submit a second report using the exact same session; observe that the system immediately intercepts the submission and displays: "Student ID verification session has already been used" (replay attack blocked).		

#	Security feature / control	Student decision Retain / Remove / Add	Reason / justification for the decision	Planned live test or evidence reference	Instructor decision Include / Exclude	Final exam result Pass / Fail / Retest
A2	**API Rate Limiting & Anti-Spam Abuse Controls:** Tiered request-rate throttlers for login, recovery, and public student reports.	**ADD**	**We are adding this feature to protect our system against brute-force login attacks and spam on public endpoints.** Using `express-rate-limit`, we implemented throttlers for: login (10 attempts / 15m), password recovery (5 attempts / 15m), public student reports (5 reports / 10m per IP), PC duplicate reports (1 report / 1m per PC), and student verification (60 requests / 10m). This stops automated abuse while keeping normal usage smooth.	**Planned Live Test:** On the staff login page (`login.html`), enter incorrect credentials and click the "Sign In" button rapidly 11 times. On the 11th click, the system locks further attempts and displays a red warning banner: "Too many login attempts. Please try again after 15 minutes" (HTTP 429 rate limit protection).		
A3	**Non-Volatile (NVS) Hardware Security Alert Persistence & Power-Cycle Resilience:** Flash incident latching across power loss.	**ADD**	**We are adding this feature to eliminate a physical tamper vulnerability where pulling the power plug clears security alarms.** In `LabSync_ESP32.ino`, unauthorized key removals and wrong-slot incidents are latched into ESP32 Preferences flash storage (`labsync_sec` namespace). If an intruder disconnects the power during an alert, our firmware immediately restores the alarm state upon boot and transmits the pending alert queue once Wi-Fi reconnects.	**Planned Live Test:** 1. Trigger an unauthorized key alarm by pulling a laboratory key from the dock without scanning; observe the onboard alarm buzzer sound. 2. Unplug the ESP32 dock's power cable (simulating cutting power to clear an alarm), wait 5 seconds, and plug it back in; observe that the firmware immediately reads flash memory on boot, restores the alarm buzzer, and syncs the buffered security alert to the MIS dashboard upon Wi-Fi reconnection.		
A4						

Instructor review flags (clarify before approval): “All routes” require sessions conflicts with the public student fault-report flow. Specify the public endpoints and their validation/abuse controls. Client auth-check.js does not replace server authorization. Polling/connection pools do not guarantee high uptime. QR token uniqueness is not proof of expiry or replay prevention. Do not add electronic locks: the stated device monitors physical keys. Source wording includes password hashes, but does not identify bcrypt despite a bcrypt-related reference.

Our Capstone Group Clarifications:

1. **Public endpoints vs. sessions:** In our system, staff routes require sessions, while student fault reporting is intentionally public at kiosks (`/verify-student-id`, `/student-cooldown`, `/submit`, `/pc-info`) with IP rate limiting (5 / 10m), PC cooldown (1 / 1m), and MySQL student cooldown (30m).
2. **Client auth-check.js vs. server auth:** `auth-check.js` is only a client-side anti-flash visual redirect. Our Express server independently validates active session roles on every API request via `requireAuth` and `requireRole`.
3. **Polling and connection pools:** 3-second polling is an interface refresh mechanism, and connection pooling manages MySQL resources. Infrastructure 99.9% uptime is managed by Railway.
4. **QR expiry & replay prevention:** Scanning student ID QR codes generates ephemeral HMAC SHA-256 tokens with 16-byte random nonces and 10-minute expiry; tokens are atomically marked `Used_At = NOW()` in MySQL upon submission, preventing replay.
5. **Physical keys (no electronic locks):** LabSync is an IoT physical key monitoring dock using ADC resistor dividers on pins D32/D33. Unauthorized key removal sounds an onboard buzzer and logs an alert. No electronic locks or solenoids are added.
6. **Password hashing algorithm:** Passwords are hashed using **bcrypt** (^6.0.0) with **12 salt rounds** (`BCRYPT_SALT_ROUNDS = 12`) in `authService.js`. Stored hashes use standard `$2b$12$...` format.

Activity assessment rubric - 30 points

Criterion	Assessment standard	Max	Score
Completeness and Accuracy of Feature Inventory	Lists the original controls accurately and correctly identifies retained, removed, and added features.	10	
Quality of Justifications	Provides clear and defensible reasons for each decision, especially requested additions or removals.	10	
Security Relevance and Technical Feasibility	Decisions align with capstone scope and identify controls that are appropriate and achievable.	5	
Clarity and Verification Readiness	Features are distinct, clearly described, and supported by a practical test or evidence reference.	5	
TOTAL		**30**	

Assess the quality of the review—not the quantity of feature changes. No additions or removals are required to earn full credit.